Contents

Genova RAG — Dashboard Functionality Documentation **2**
Document Metadata . . . 2
Table of Contents . . . 3
Executive Summary . . . 3
Dashboard Scope . . . 3
Authentication and Authorization . . . 4
Login requirements . . . 4
Authentication middleware . . . 4
Session / token handling . . . 4
User identification . . . 4
Role-based access . . . 4
Permission checks . . . 4
Protected API endpoints . . . 5
Unauthorized behavior . . . 5
Session expiration . . . 5
Relevant security configuration . . . 5
Database Functionality . . . 5
Database technology . . . 5
Relevant tables / collections . . . 5
Operations . . . 6
Complete data lifecycle . . . 6
D. Vector database . . . 7
E. LLM generation . . . 7
F. RAG execution flow . . . 7
RAG Architecture and Functionality . . . 8
A. RAG entry point . . . 8
B. Retrieval pipeline . . . 8
API: GET /api/v1/chat/sessions . . . 9
API: POST /api/v1/chat/sessions . . . 9
API: PUT /api/v1/chat/sessions/{id} . . . 9
API: DELETE /api/v1/chat/sessions/{id} . . . 10
API: GET /api/v1/chat/history/session/{id} . . . 10
API: POST /api/v1/chat/ask/stream . . . 10
API: WS /api/v1/chat/ws . . . 11
API: GET /api/v1/auth/me . . . 11
API: POST /api/v1/auth/dev-login . . . 12
API: POST /api/v1/auth/refresh . . . 12
API: POST /api/v1/auth/logout . . . 12
API: PUT /api/v1/auth/hf-token . . . 13
API: GET /api/v1/auth/google-drive/status . . . 13
API: POST /api/v1/auth/google-drive/connect . . . 13
API: DELETE /api/v1/auth/google-drive/disconnect . . . 13
API: PATCH /api/v1/documents/{document_id} . . . 14
API: DELETE /api/v1/documents/{document_id} . . . 14
API: POST /api/v1/documents/{document_id}/restore . . . 14
API: DELETE /api/v1/documents/{document_id}/permanent . . . 15
API: GET /api/v1/documents/trash . . . 15
API: POST /api/v1/documents/{document_id}/chunk_settings . . . 15
API: GET /api/v1/documents/{document_id}/pdf . . . 16
API: POST /api/v1/documents/upload . . . 16
API Documentation . . . 17
API: GET /api/v1/documents/ . . . 17
Backend Dependency Table . . . 17
Backend Architecture . . . 19
Entry point . . . 19

Authentication dependency . . . 19
Route handlers used by the dashboard . . . 20
Services and business logic invoked . . . 20
Frontend State Management . . . 21
Page-local state (DashboardPage) . . . 21
Data fetching . . . 21
Authentication context . . . 21
CompareView elements . . . 22
Header elements (dashboard shell) . . . 22
Empty / error / loading states (verified) . . . 22
User Interactions . . . 22
1. Open dashboard . . . 22
2. Load document library . . . 22
3. Upload a document . . . 23
4. Select a document . . . 23
5. Ask a question (RAG chat) . . . 23
6. Open citation in PDF . . . 23
7. Rename / delete / restore document . . . 23
8. Change chat session . . . 23
ChatPanel elements . . . 24
PDFViewer elements . . . 24
DocumentSidebar elements . . . 24
ChatSessionSidebar elements . . . 25
UI Functionality . . . 25
Page-level elements (DashboardPage) . . . 25
Component hierarchy . . . 26
Frontend file inventory . . . 26
Frontend Architecture . . . 27
Route and page structure . . . 27

Genova RAG — Dashboard Functionality Documentation

Page: http://localhost:3000/dashboard
Application: Genova Document Assistant (PDF-Assistant-RAG-dev)
Author: Senior Software Architecture / Documentation Engineering

---

Document Metadata

Field	Value
Document title	Dashboard Functionality Documentation — `/dashboard`
Target page	http://localhost:3000/dashboard
Product	Genova RAG — Document Intelligence / RAG Assistant
Version	1.0
Status	Final
Scope	Frontend page, backend APIs, services, database, RAG pipeline, authentication, error handling, configuration
Source of truth	Inspected source code of `frontend/` and `backend/` directories
Related files	`docs/dashboard-functionality.css`, `docs/README-dashboard-docs.md`

Table of Contents

1. Executive Summary
2. Dashboard Scope
3. Technology Stack
4. Frontend Architecture
5. UI Functionality
6. Frontend State Management
7. Frontend File Inventory
8. Backend Architecture
9. Backend Dependency Table
10. API Documentation
11. RAG Architecture and Functionality
12. Database Functionality
13. Authentication and Authorization
14. External Services and Dependencies
15. Function-Level Documentation
16. Complete Dashboard Execution Flow
17. Mermaid Architecture Diagrams
18. Error Handling
19. Configuration and Environment
20. Dependency and Impact Analysis
21. Limitations
22. Verification Notes

Executive Summary

The `/dashboard` page (the "Genova desk") is the primary user interface of the Genova RAG application. It is a Next.js 16 client page served at http://localhost:3000/frontend and backed by a FastAPI Python service at http://localhost:7860/api/v1.

The dashboard combines functional areas in a single desk layout: document library, PDF viewer, RAG chat, chat session management, notes, compare view, Google Drive status, and authentication UI. Document and chat resources are scoped to the authenticated user (`user_id` ownership filter on every query).

Dashboard Scope

In scope (verified in source code):

- Route `frontend/src/app/dashboard/page.tsx` and its child components
- All API endpoints invoked from the dashboard components and their Zustand stores
- Backend handlers for those endpoints in `backend/app/routes/documents.py`, `backend/app/routes/chat.py`, `backend/app/routes/auth.py`
- Services, RAG pipeline, retrieval, embeddings, vector store, LLM clients invoked by those handlers
- Database tables read or written by those handlers: `users`, `documents`, `chat_messages`, `chat_sessions` (plus `drive_connections` for the Drive status UI)
- Authentication/authorization middleware protecting dashboard APIs
- Configuration variables traced to dashboard behavior

Out of scope (documented as absent/not part of dashboard):

- `/admin`, `/settings`, `/share`, `/drive`, `/login`, `/register`, `/verify-email`, `/auth/*` pages except where their APIs are called from the dashboard
- Background ingestion workers (Celery) except where triggered by dashboard upload
- GraphRAG, quiz generation, batch upload, URL upload, email service, audit logging, observability/metrics, cleanup scheduler — present in the repository but not invoked by the dashboard page code inspected

Authentication and Authorization

Login requirements

- **Dashboard gate:** `DashboardPage` mounts `useAuth()`; while `initialized === false` or no user, the page renders `null` (no UI shown).
- **Dev auto-login:** Enabled when `NEXT_PUBLIC_DEV_AUTH_BYPASS !== "false"` (frontend `.env.local`) AND `DEV_AUTH_BYPASS=true` with `ENVIRONMENT != "production"` (backend config). On localhost, this auto-signs in via `POST /api/v1/auth/dev-login`, returning tokens stored in `localStorage` + HttpOnly cookies.
- **OAuth login:** Google OAuth flow (`/api/v1/auth/google`) and HuggingFace OAuth (`/api/v1/auth/huggingface`) complete external auth and return JWT tokens; frontend sets cookies + `localStorage` via `auth-store`.
- **Login form:** `/login` page is separate; the dashboard itself does not render a login form — the auth gate either auto-signs in or redirects.

Authentication middleware

- `get_current_user` (`backend/app/auth.py`) — FastAPI `Depends` on every dashboard route handler. Extracts JWT from `Authorization: Bearer <token>` header or `access_token` HttpOnly cookie. Rejects if no token present (401). Decodes HS256 via `decode_token`. Loads `User` from DB by `sub` (user_id). Sets `request.state.user_id` and observability context var `user_id_var`.
- `HTTPBearer(auto_error=False)` — allows routes to opt out; dashboard routes always require auth.
- **API key auth** — tokens starting `pdf_rag_` are accepted (hashed SHA-256 lookup against `ApiKey` table).

Session / token handling

- Access token: stored in `localStorage` (`api.getToken()`) + sent as `Authorization: Bearer <token>` on every API call.
- Refresh token: stored in `localStorage`; automatically exchanged by `ApiClient.tryRefreshToken()` on 401 responses (POST to `/api/v1/auth/refresh`).
- HttpOnly cookies: `access_token` and `refresh_token` set by backend on login/dev-login; `secure=True` only in production; `samesite="lax"`.
- Token refresh flow: dashboard request → 401 → `ApiClient` attempts refresh via `tryRefreshToken()` → retry original request with new token. If refresh fails → frontend forces logout (clears `localStorage`, resets auth-store, router.replace to `/login`).

User identification

- Every dashboard API call carries the authenticated user's `user_id` (from JWT `sub`).
- All document queries filter `Document.user_id == user_id`; all chat messages have `user_id` set.
- The `Header` component shows the username and an "HF Token" management dialog.
- Admin-only routes (`get_admin_user`) are **not** used by the dashboard; document/chats are owner-scoped only.

Role-based access

- **User role:** `UserRole.user` or string `"user"` — can upload documents, chat with RAG, view own library, manage own sessions, manage own Drive connection.
- **Admin flag:** `User.is_admin` boolean; also `User.role == UserRole.admin`. The dashboard does **not** expose admin-only features; `get_admin_user` dependency is not wired into any dashboard route.
- **Drive connection:** per-user; no cross-user access.

Permission checks

- Each dashboard-backed route handler enforces `Document.user_id == user.id` (or equivalent for chat sessions).
- The `get_current_user` dependency raises `HTTPException(401)` if no valid token; `ForbiddenException(403)` if user lacks role/admin (not used in dashboard routes).
- Frontend UI disables actions when unauthenticated (e.g., upload/button hidden until `initialized`).

Protected API endpoints

Every endpoint listed in the API Documentation section is protected by `get_current_user`. The dashboard does not call any unprotected endpoints (aside from the dev-login bypass which is explicitly scoped to localhost).

Unauthorized behavior

- **No token / invalid token:** `401 Unauthorized` → `ApiClient.handleUnauthorized()` → frontend clears `localStorage` tokens, resets auth-store (`user = null, token = null, initialized = true`), router.replace to `/login`.
- **Expired token:** Same as no token — refresh attempted first; if refresh fails → logout.
- **Session expired:** User re-authenticates via dev-login (localhost) or Google/HF OAuth.

Session expiration

- Access token expiry: 15 minutes (`JWT_ACCESS_EXPIRY_MINUTES = 15` in config).
- Refresh token expiry: 7 days (`JWT_REFRESH_EXPIRY_DAYS = 7`).
- Frontend fail-safe: `auth-store.initializeAuth()` sets a `window.setTimeout` fail-safe of ~30s; if `initialized` not true by then, page proceeds on anonymous state to prevent infinite loading.

Relevant security configuration

Variable	Purpose	Source
`DEV_AUTH_BYPASS`	Allow localhost auto-sign-in; forced OFF in production	`backend/app/config.py` default `True`; overridden in docker `environment`
`ENVIRONMENT`	`development` / `production`; controls validate_production checks	`backend/app/config.py` default `"development"`
`SECRET_KEY`	JWT signing key; must be 32 chars in production	`backend/app/config.py` default `"change-me-in-production-please"`
`JWT_ALGORITHM`	Only HS256 allowed	`backend/app/config.py` default `"HS256"`
`JWT_ACCESS_EXPIRY_MINUTES`	Access token TTL	`backend/app/config.py` default `15`
`JWT_REFRESH_EXPIRY_DAYS`	Refresh token TTL	`backend/app/config.py` default `7`
`NEXT_PUBLIC_DEV_AUTH_BYPASS`	Frontend gate for dev bypass	`frontend/.env.local` default `true`
`ALLOWED_ORIGINS`	CORS allowlist (includes http://localhost:3000)	`backend/app/config.py` default `"http://localhost:3000,http://localhost:7`

Database Functionality

Database technology

- **Primary:** SQLAlchemy (sync) with SQLite default (`DATABASE_URL = "sqlite:///./data/app.db"`).
- **Production alternative:** PostgreSQL+psycopg via docker-compose or env.
- **Migrations:** `init_db()` in `backend/app/database.py` — `Base.metadata.create_all(bind=engine)` + raw-SQL migrations.

Relevant tables / collections

Entity	Storage	Key fields (dashboard-relevant)	Relationships
`User`	SQLite / Postgres	`id`, `username`, `email`, `is_admin`, `is_deleted`, `drive_folder_id`, `last_login`, `hf_token` (encrypted)	— owns — `documents`, `chat_messages`, `chat_sessions`, `drive_connections`
`Document`	SQLite / Postgres	`id`, `user_id`, `original_name`, `filename`, `file_size`, `page_count`, `chunk_count`, `status`	`user_id` → `User.id`; one → many `chat_messages`
`ChatSession`	SQLite / Postgres	`id`, `title`, `created_at`	one → many `chat_messages`
`ChatMessage`	SQLite / Postgres	`id`, `user_id`, `document_id`, `session_id`, `role`, `content`, `sources_json`	`user_id` → `User.id`; `document_id` → `Document.id`; `session_id` → `ChatSession.id`
`DriveConnection`	SQLite / Postgres	`id`, `user_id`, `folder_id`, `credentials_json`	`user_id` → `User.id`
ChromaDB	Persistent client	Per-user collection `user_<id>`; chunk metadata + embedding vectors	User isolation

Operations

- **Read:** `GET /api/v1/documents/` → user documents; `GET /api/v1/chat/sessions`; `GET /api/v1/chat/history/sessio` `GET /api/v1/documents/{id}/pdf`.
- **Write:** `POST /api/v1/documents/upload` → create row + enqueue ingestion; `PATCH /api/v1/documents/{id}` → rename; `DELETE /api/v1/documents/{id}` → soft-delete; `POST /api/v1/documents/{id}/restore`; `DELETE /api/v1/documents/{id}/permanent` → hard-delete + Chroma cleanup; `POST /api/v1/chat/sessions`; `PUT /api/v1/chat/sessions/{id}`; `DELETE /api/v1/chat/sessions/{id}`; `POST /api/v1/chat/ask/stream` → persist user message + RAG pipeline.
- **Transactions:** `get_db_session()` context manager (commit on success, rollback on SQLAlchemyError → `AppException`).
- **Indexes:** Confirmed via migration: `ix_users_email`, `ix_documents_user_id`, `ix_documents_is_deleted`, `ix_chat_messages_user_id`, `ix_chat_messages_document_id`, `ix_chat_messages_created_at`, `ix_drive_connections`
- **Chroma indexes:** per-user collection names; cosine HNSW index on vectors.

Complete data lifecycle

1. **Upload** → Frontend `POST /api/v1/documents/upload` → Backend `Document` row (`status = "queued"`) + Celery task → `ingest_document()` → chunks → `store_chunks` (embed → Chroma `collection.add`) + BM25 rebuild → update `Document` row (`status = "ready"`, counts, summary, keywords, URLs, `completed_at`).
2. **Question** → Frontend `POST /api/v1/chat/ask/stream` → Backend validate + persist user message + cache check → `generate_answer_stream` → `retrieve` (embed → Chroma + BM25 → RFF → sort/confidence) → LLM chat completion (Ollama/Bedrock) → SSE stream → frontend renders + saves assistant `ChatMessage` row.
3. **Rename / Trash / Restore / Permanent delete** → SQL `UPDATE` / `DELETE` + Chroma row cleanup (`delete_document_chunks`).
4. **Session switch** → `GET /api/v1/chat/history/session/{id}` → load messages → replace ChatPanel.
5. **Auth** → JWT in localStorage → `get_current_user` per-request → per-document `user_id` filter on every read. ### C. Embedding functionality

- **Provider:** Ollama (default), via `OllamaEmbeddings` (`backend/app/rag/llm/ollama_client.py`) calling `POST http://localhost:11434/api/embed`.
- **Model:** `nomic-embed-text` (config `EMBEDDING_MODEL`); HuggingFace fallback via `EMBEDDING_PROVIDER=huggingface`.
- **Dimensions:** 768 (`EMBEDDING_DIMENSION=768` for nomic; 384 if HF MiniLM).
- **Query embedding function:** `embed_query(query)` in `backend/app/rag/embeddings/embeddings.py` — cache key = SHA-256 of query; LRU cache (512 entries) or Redis (`REDIS_URL`); cache TTL 86400s; prefixes `search_query:` for nomic before the Ollama call.
- **Document embedding function:** Index-time path: `get_embedding_model().embed_documents(texts)` inside `store_chunks()` (`vectorstore.py`), prefixing `search_document:` for nomic.
- **Input preprocessing:** None beyond provider-specific prefix.
- **Embedding storage:** ChromaDB collection `user_<sanitized_user_id>`; metadata includes `filename`, `document_id`, `page`, `chunk_type`, `bbox`.
- **Embedding retrieval:** `query_chunks` → Chroma `collection.query(query_embeddings=[...], n_results=..., where={...})`; score = `round(1 - distance, 4)`.

D. Vector database

- **Database:** ChromaDB (`chromadb.PersistentClient`).
- **Storage location:** `settings.CHROMA_PERSIST_DIR` (default `./data/chroma_db_ollama_nomic`; docker: `/app/data/chroma_db`).
- **Collections:** One per user: `get_collection_name(user_id)` → `user_` + user id with hyphens replaced by underscores, truncated to 63 chars.
- **Metadata:** `filename`, `document_id`, `page`, `chunk_type`, `bbox`, plus embedding vectors.
- **Similarity metric:** Cosine (`hnsw:space = "cosine"`); frontend-facing score = `round(1 - distance, 4)`.
- **Search implementation:** `query_chunks(...)` → `collection.query` with `where` filter; parallel BM25 via `app/rag/retrieval/bm25.py`.
- **Persistence:** PersistentClient (on-disk); survives restarts.
- **Filtering:** `where={"document_id": {"$eq": document_id}}` for single-document queries; no filter for multi-session queries.

E. LLM generation

- **Provider:** Ollama (default, `settings.LLM_PROVIDER == "ollama"`) or AWS Bedrock (`"bedrock"`); HuggingFace ReAct agent available but not the default dashboard path.
- **Model:** `qwen2.5:3b` (`LLM_MODEL`), temperature 0.3, max new tokens 1024. Bedrock: `us.amazon.nova-2-lite-v1:0` (`BEDROCK_MODEL_ID`, region `us-east-1`).
- **Configuration:** `backend/app/config.py` (`LLM_PROVIDER`, `LLM_MODEL`, `LLM_TEMPERATURE`, `LLM_MAX_NEW_TOKENS`, `OLLAMA_BASE_URL=http://localhost:11434`, `BEDROCK_MODEL_ID`, `BEDROCK_REGION`); frontend Header lets user pick `ollama` vs `bedrock` via a `<select>`.
- **Prompt construction:** `_format_context(chunks)` joins retrieved chunks; system prompt `SYSTEM_PROMPT` + `RAG_PROMPT_TEMPLATE` (in `backend/app/rag/prompts/prompts.py`) instruct model to answer from context and cite sources. Quiz-intent and greeting branches bypass retrieval.
- **Retrieved context:** Top 8 chunks with text, filename, page, score, confidence.
- **Generation function:** `OllamaChatClient.chat_completion(messages, model, max_tokens, temperature, stream)` → POST `http://localhost:11434/api/chat`.
- **Output format:** Streaming text chunks; SSE events `sources` → `token*` → `usage` → `done`.
- **Response parsing:** Frontend parses `data:` lines; assistant message saved with `sources_json`.
- **Retry behavior:** Celery autoretries for ingestion; LLM calls have no application-level retry (Ollama errors surface as SSE `error` event / 502).
- **Error handling:** `validate_user_input` blocks unsafe prompts (`UnsafePromptException`, 400); Ollama unreachable → `ExternalServiceException` (502); empty retrieval → answer without sources.

F. RAG execution flow

```
User Query
  → ChatPanel (frontend/src/components/chat/ChatPanel.tsx, handleSend)
  → api.streamPost("/api/v1/chat/ask/stream", {question, document_id, top_k, llm_provider})
```

```
  (frontend/src/lib/api.ts)
→ POST /api/v1/chat/ask/stream (backend/app/routes/chat.py, ask_question_stream)
  → validate_user_input (app/rag/utils/security.py)
  → save user message (ChatMessage row)
  → cache check (app/cache.py get_cached_payload)
  → generate_answer_stream(...) (app/rag/llm/agent.py)
    → retrieve(question, user_id, document_id, top_k) (app/rag/retrieval/retriever.py)
      → embed_query(...) (app/rag/embeddings/embeddings.py → Ollama /api/embed)
      → query_chunks(...) (app/rag/vectordb/vectorstore.py → Chroma cosine search)
      → query_bm25(...) (app/rag/retrieval/bm25.py)
      → rrf_merge(...) → sort → top_k chunks + confidence
    → _format_context(chunks)
    → SYSTEM_PROMPT + RAG_PROMPT_TEMPLATE + question
    → OllamaChatClient.chat_completion(stream) (app/rag/llm/ollama_client.py → Ollama /api/chat)
  → SSE events: sources, token, usage, done
  → save assistant message (sources_json)
→ Backend HTTP response (text/event-stream)
→ ChatPanel renders streaming tokens + source chips
→ Citation click → PDFViewer opens GET /api/v1/documents/{id}/pdf?token= at page + highlight rect
```

RAG Architecture and Functionality

The dashboard **does** invoke the RAG pipeline: when the user sends a question in `ChatPanel`, the frontend calls `POST /api/v1/chat/ask/stream`, which executes the full retrieve-then-generate path.

A. RAG entry point

1. **Trigger:** User types a question in `ChatPanel` (`frontend/src/components/chat/ChatPanel.tsx`) and presses Enter or clicks Send (`handleSend`).
2. **Frontend function:** `handleSend()` appends an optimistic user message, then calls `api.streamPost("/api/v1/chat/ask/ { question, document_id, document_ids, top_k, llm_provider })`. A WebSocket at `/api/v1/chat/ws` is attempted first; on failure it falls back to SSE.
3. **API endpoint:** `POST /api/v1/chat/ask/stream` (`backend/app/routes/chat.py`, handler `ask_question_stream`).
4. **Backend handler:** `ask_question_stream` validates input, persists the user message, checks the response cache, then streams `generate_answer_stream(...)` results as SSE.
5. **RAG service entry point:** `generate_answer_stream()` in `backend/app/rag/llm/agent.py`, delegating to `_generate_answer_stream_direct(...)` for Ollama/Bedrock providers.

B. Retrieval pipeline

Implemented steps (verified in `backend/app/rag/retrieval/retriever.py`):

1. **Query processing** — `transform_query(query)` produces the original question plus up to 3 paraphrase variants (LLM-assisted rewrite).
2. **Query embedding** — `embed_query(search_query)` for each variant (Ollama `nomic-embed-text`, 768-d; see section C).
3. **Vector database search** — `query_chunks(query_embedding=..., user_id=..., document_id=..., top_k=TOP_K_RETRIEVAL=20)` against ChromaDB with cosine similarity (`hnsw:space = "cosine"`), filtered by `document_id` when a single document is selected.
4. **Keyword search** — `query_bm25(...)` in parallel (BM25 index rebuilt per document at ingestion).
5. **Hybrid retrieval** — `rrf_merge(vector_results, bm25_results, k=RRF_K=60)` fuses the two ranked lists via Reciprocal Rank Fusion; the fused score becomes `score`.
6. **Metadata filtering** — Chroma `where` filter on `document_id` (single-doc scope); user isolation via per-user Chroma collections (`user_<id-with-underscores>`).
7. **Reranking** — `get_reranker()` returns a cross-encoder only if `RERANKER_ENABLED` is true; default is **False**, so the pipeline sorts by `score` and slices `top_k` (default 8, `TOP_K_RERANK`).
8. **Chunk selection** — `_merge_candidates()` deduplicates; final top chunks receive a normalized `confidence` percentage (0–100 relative to max score).

9. **Context construction** — `_format_context(chunks)` in `agent.py` builds the context string; `SYSTEM_PROMPT` + `RAG_PROMPT_TEMPLATE` wrap it with the user question.

Steps absent: no query expansion beyond paraphrases, no query decomposition, no long-context re-ranking beyond the optional cross-encoder, no agentic tool use in the default Ollama direct path.

API: GET /api/v1/chat/sessions

Method: GET

Endpoint: `/api/v1/chat/sessions`

Frontend caller: `chat-store.fetchSessions()` — `frontend/src/store/chat-store.ts`

Backend handler: `list_sessions` — `backend/app/routes/chat.py`

Purpose: List the user's chat sessions. On first load, if sessions exist, the first session is selected and its history fetched.

Authentication: Required.

Request: No request body.

Response: `ChatSessionResponse[]` — `[{ id, title, created_at }]`.

Error responses: Frontend catches and gracefully ignores if endpoint unavailable (CI/tests note in code).

API: POST /api/v1/chat/sessions

Method: POST

Endpoint: `/api/v1/chat/sessions`

Frontend caller: `chat-store.createSession(title)` — `frontend/src/store/chat-store.ts`

Backend handler: `create_session` — `backend/app/routes/chat.py`

Purpose: Create a new chat session and set it active.

Authentication: Required.

Request body:

```
{
  "title": "Chat 1"
}
```

Response: `ChatSessionResponse` — `{ id, title, created_at }`.

Error responses: 401 unauthenticated; 400 validation.

API: PUT /api/v1/chat/sessions/{id}

Method: PUT

Endpoint: `/api/v1/chat/sessions/{id}`

Frontend caller: `chat-store.renameSession(id, title)` — `frontend/src/store/chat-store.ts`

Backend handler: `rename_session` — `backend/app/routes/chat.py`

Purpose: Rename a chat session.

Authentication: Required.

Request body:

```
{
  "title": "New title"
}
```

Response: Updated `ChatSessionResponse`.

Error responses: 401, 404.

API: DELETE /api/v1/chat/sessions/{id}

Method: DELETE

Endpoint: `/api/v1/chat/sessions/{id}`

Frontend caller: `chat-store.deleteSession(id)` — `frontend/src/store/chat-store.ts`

Backend handler: `delete_session` — `backend/app/routes/chat.py`

Purpose: Delete a chat session; frontend treats "not found" as success (idempotent).

Authentication: Required.

Request: No request body.

Response: 204 (no content).

Error responses: 401, 404 (tolerated by frontend).

API: GET /api/v1/chat/history/session/{id}

Method: GET

Endpoint: `/api/v1/chat/history/session/{id}`

Frontend caller: `chat-store.fetchSessionHistory(id)` — `frontend/src/store/chat-store.ts`

Backend handler: `get_session_history` — `backend/app/routes/chat.py`

Purpose: Return all messages for a session, used to restore the chat view when switching sessions.

Authentication: Required.

Request: No request body.

Response: `ChatHistoryResponse` — `{ messages: ChatMessageResponse[], document_id?: string }`.

Each message: `id`, `role`, `content`, `sources: SourceChunk[]`, `attachments`, `feedback`, `created_at`, `usage`, `llm_provider`, `response_time_ms`, `quiz`.

API: POST /api/v1/chat/ask/stream

Method: POST

Endpoint: `/api/v1/chat/ask/stream`

Frontend caller: `ChatPanel.handleSend()` → `api.streamPost("/api/v1/chat/ask/stream", body)` — `frontend/src/components/chat/ChatPanel.tsx`

Backend handler: `ask_question_stream` — `backend/app/routes/chat.py`

Purpose: Submit a question for RAG answering and stream the response as Server-Sent Events. (A WebSocket at `/api/v1/chat/ws` is attempted first in ChatPanel; on failure the page falls back to this SSE endpoint.)

Authentication: Required.

Authorization: Document ownership checked when `document_id` is provided (`Document.id == payload.document_id AND Document.user_id == user.id`).

Request body (`ChatRequest` schema):

```
{
  "question": "Does the label recommend every-2-week dosing?",
  "document_id": "optional-single-document-id",
  "document_ids": ["optional", "multi-doc ids"],
```

```
  "top_k": 8,
  "llm_provider": "ollama"
}
```

Response: `text/event-stream` SSE stream. Events (verified in `event_stream` and frontend parser):

- `data: { "type": "sources", "data": [ { text, filename, page, score?, confidence?, highlightRects? }, ...] }`
- `data: { "type": "token", "data": "partial answer text..." }` (repeated)
- `data: { "type": "usage", "data": { input_tokens, output_tokens, total_tokens, estimated_cost_usd, model_id, provider } }`
- `data: { "type": "done" }`
- `data: { "type": "error", "data": "..." }` (on failure)

Error responses: 400 unsafe prompt / validation; 401 unauthenticated; 404 document not found; 429 rate limited (`CHAT_QUERY_RATE_LIMIT = 15/minute`); 502 LLM service error.

Execution flow: Frontend `ChatPanel` → SSE stream → `ask_question_stream` → save user message → cache check → `generate_answer_stream` → `retrieve` (embed → Chroma + BM25 → RRF → rerank) → `_format_context` → LLM chat completion → SSE tokens → save assistant message → frontend renders stream.

API: WS /api/v1/chat/ws

Method: WebSocket

Endpoint: `/api/v1/chat/ws`

Frontend caller: `ChatPanel` — attempts WebSocket connection first, then falls back to `POST /api/v1/chat/ask/stream`.

Backend handler: `chat_websocket` — `backend/app/routes/chat.py`

Purpose: Real-time chat transport alternative to SSE. Rate-limited manually (`check_chat_ws_rate_limit`, 15/min).

Authentication: Required via `?token=` query parameter.

Note: The dashboard's primary verified path is SSE; WebSocket is a fallback transport attempted in `ChatPanel`.

API: GET /api/v1/auth/me

Method: GET

Endpoint: `/api/v1/auth/me`

Frontend caller: `auth-store.initializeAuth()` — `frontend/src/store/auth-store.ts`

Backend handler: `read_users_me` — `backend/app/routes/auth.py`

Purpose: Resolve the authenticated user from the stored access token and return the user profile.

Authentication: Required (Bearer token passed in `api.get` options).

Request: No request body.

Response: `UserResponse` — `id`, `username`, `email`, `is_admin`, `hf_token` (optional), `created_at`.

```
{
  "id": "string",
  "username": "user",
  "email": "user@example.com",
  "is_admin": false,
  "hf_token": null,
  "created_at": "2026-01-01T00:00:00Z"
}
```

Error responses: 401 invalid/expired token (falls back to dev-login or anonymous state in the frontend).

API: POST /api/v1/auth/dev-login

Method: POST

Endpoint: `/api/v1/auth/dev-login`

Frontend caller: `auth-store.initializeAuth()` (dev bypass branch) — `frontend/src/store/auth-store.ts`

Backend handler: `dev_login` — `backend/app/routes/auth.py`

Purpose: Localhost-only auto sign-in: creates or retrieves a dev user and returns access + refresh tokens. Enabled when `NEXT_PUBLIC_DEV_AUTH_BYPASS !== "false"` on the frontend and `DEV_AUTH_BYPASS=true` with `ENVIRONMENT != production` on the backend.

Authentication: None (self-authenticating dev endpoint).

Request body: `{}` (empty).

Response: `TokenResponse` — `access_token`, `refresh_token`, `token_type: "bearer"`, `user: UserResponse`.

Error responses: 400/500 if backend bypass disabled.

API: POST /api/v1/auth/refresh

Method: POST

Endpoint: `/api/v1/auth/refresh`

Frontend caller: `ApiClient.tryRefreshToken()` — `frontend/src/lib/api.ts` (internal, on 401)

Backend handler: `refresh_token` — `backend/app/routes/auth.py`

Purpose: Exchange a valid refresh token for a new access token (and optional rotated refresh token). Used automatically by the API client when a dashboard request returns 401.

Authentication: Required — valid `refresh_token` in body.

Request body:

```
{
  "refresh_token": "string"
}
```

Response: `{ access_token: string, refresh_token?: string }`.

Error responses: 401 if refresh invalid/expired → frontend forces logout.

API: POST /api/v1/auth/logout

Method: POST

Endpoint: `/api/v1/auth/logout`

Frontend caller: `auth-store.logout()` / `Header.handleLogout()` — `frontend/src/store/auth-store.ts`, `frontend/src/components/layout/Header.tsx`

Backend handler: `logout` — `backend/app/routes/auth.py`

Purpose: Clear session cookies (HttpOnly `access_token`/`refresh_token`). Frontend also clears `localStorage` and resets auth state regardless of API outcome.

Authentication: Optional (errors swallowed on frontend).

Request: No request body.

Response: `{ message: "Successfully logged out" }`.

API: PUT /api/v1/auth/hf-token

Method: PUT

Endpoint: `/api/v1/auth/hf-token`

Frontend caller: `Header` HF-token dialog / `auth-store.setHfToken()` — `frontend/src/components/layout/Header.tsx`, `frontend/src/store/auth-store.ts`

Backend handler: `update_hf_token` — `backend/app/routes/auth.py`

Purpose: Save or remove the user's HuggingFace access token (encrypted at rest via `EncryptedString`). Used for gated HF models.

Authentication: Required.

Request body: `{ "hf_token": "hf_..." }` or `{ "hf_token": "" }` to remove.

Response: `UserResponse` (updated user record).

Error responses: 401 unauthenticated; 400 validation.

API: GET /api/v1/auth/google-drive/status

Method: GET

Endpoint: `/api/v1/auth/google-drive/status`

Frontend caller: `DocumentSidebar` (`useEffect` on mount) — `frontend/src/components/document/DocumentSidebar.tsx`

Backend handler: `google_drive_status` — `backend/app/routes/auth.py`

Purpose: Return whether the user has a connected Google Drive folder.

Authentication: Required.

Request: No request body.

Response: `GoogleDriveStatusResponse` — `{ connected: boolean }`.

API: POST /api/v1/auth/google-drive/connect

Method: POST

Endpoint: `/api/v1/auth/google-drive/connect`

Frontend caller: `DocumentSidebar` Drive connect button.

Backend handler: `google_drive_connect` — `backend/app/routes/auth.py`

Purpose: Initiate Google Drive OAuth connection.

Authentication: Required.

Request: No request body (frontend initiates OAuth flow).

Response: Status/URL payload per handler.

API: DELETE /api/v1/auth/google-drive/disconnect

Method: DELETE

Endpoint: `/api/v1/auth/google-drive/disconnect`

Frontend caller: `DocumentSidebar` Drive disconnect button.

Backend handler: `google_drive_disconnect` — `backend/app/routes/auth.py`

Purpose: Remove the user's Drive connection record.

Authentication: Required.

Request: No request body.

Response: `GoogleDriveStatusResponse` — `{ connected: false }`.

API: PATCH /api/v1/documents/{document_id}

Method: PATCH

Endpoint: `/api/v1/documents/{document_id}`

Frontend caller: `DocumentSidebar` rename handler → `api.renameDocument(documentId, name)` — `frontend/src/lib/api.ts`

Backend handler: `update_document` — `backend/app/routes/documents.py`

Purpose: Rename a document (updates `original_name`). Ownership enforced.

Authentication: Required.

Authorization: `Document.user_id == user.id` and not soft-deleted.

Request body:

```
{
  "original_name": "renamed-document.pdf"
}
```

Response: `DocumentResponse` — updated document record.

Error responses: 404 not found, 401 unauthenticated.

API: DELETE /api/v1/documents/{document_id}

Method: DELETE

Endpoint: `/api/v1/documents/{document_id}`

Frontend caller: `DocumentSidebar` trash handler → `api.delete("/api/v1/documents/" + docId)` — `frontend/src/components/document/DocumentSidebar.tsx`

Backend handler: `delete_document` — `backend/app/routes/documents.py`

Purpose: Soft-delete a document: sets `is_deleted = true`, `deleted_at`. Document disappears from the active library and moves to Trash.

Authentication: Required.

Authorization: Owner-scoped.

Request: No request body.

Response: 200 with message object (`{"message": "Document moved to trash"}` shape per handler return).

Error responses: 404 not found, 401 unauthenticated.

API: POST /api/v1/documents/{document_id}/restore

Method: POST

Endpoint: `/api/v1/documents/{document_id}/restore`

Frontend caller: `DocumentSidebar` trash-tab restore handler → `api.post("/api/v1/documents/" + docId + "/restore")`

Backend handler: `restore_document` — `backend/app/routes/documents.py`

Purpose: Restore a soft-deleted document to the active library (`is_deleted = false`).

Authentication: Required. Authorization: owner-scoped.

Request: No request body.

Response: `DocumentResponse` — restored document record.

Error responses: 404 not found, 401 unauthenticated.

API: DELETE /api/v1/documents/{document_id}/permanent

Method: DELETE

Endpoint: `/api/v1/documents/{document_id}/permanent`

Frontend caller: `DocumentSidebar` trash-tab "delete forever" handler → `api.delete("/api/v1/documents/" + docId + "/permanent")`

Backend handler: `permanent_delete_document` — `backend/app/routes/documents.py`

Purpose: Permanently delete a document: DB row removed and vector/BM25 chunks deleted via `delete_document_chunks(document_id, user_id)`.

Authentication: Required. Authorization: owner-scoped.

Request: No request body.

Response: 200 with confirmation message.

Error responses: 404 not found, 401 unauthenticated.

API: GET /api/v1/documents/trash

Method: GET

Endpoint: `/api/v1/documents/trash`

Frontend caller: `DocumentSidebar` trash tab → `api.get("/api/v1/documents/trash")`

Backend handler: `list_trash_documents` — `backend/app/routes/documents.py`

Purpose: List soft-deleted documents belonging to the user.

Authentication: Required. Authorization: `user_id` filter + `is_deleted == true`.

Request: No request body.

Response: `DocumentListResponse` — `{ documents: DocumentResponse[], total: int }`.

Error responses: 401 unauthenticated; 500 database error.

API: POST /api/v1/documents/{document_id}/chunk_settings

Method: POST

Endpoint: `/api/v1/documents/{document_id}/chunk_settings`

Frontend caller: `DocumentSettings` modal save → `api.post("/api/v1/documents/" + document.id + "/chunk_settings", { chunk_size, chunk_overlap })` — `frontend/src/components/document/DocumentSettings.tsx`

Backend handler: `update_chunk_settings` — `backend/app/routes/documents.py`

Purpose: Update a document's chunk size/overlap, reset status to `pending`, clear counts/summary, and re-queue ingestion with new settings.

Authentication: Required. Authorization: owner-scoped.

Request body:

```
{
  "chunk_size": 1000,
  "chunk_overlap": 200
}
```

Response: `DocumentResponse` with `task_id` of the re-ingestion job.

Error responses: 400 validation (overlap >= size), 404 not found, 401 unauthenticated.

API: GET /api/v1/documents/{document_id}/pdf

Method: GET

Endpoint: `/api/v1/documents/{document_id}/pdf`

Frontend caller: `PDFViewer` via `api.getPdfUrl(documentId)` → `${API_BASE}/api/v1/documents/${documentId}/pdf?toke` — `frontend/src/lib/api.ts` and `frontend/src/components/document/PDFViewer.tsx`

Backend handler: `serve_pdf` — `backend/app/routes/documents.py`

Purpose: Serve the stored PDF file bytes so `react-pdf` can render it in the dashboard viewer. Token passed as query parameter for the PDF.js request.

Authentication: Required (token validated against stored file access; ownership enforced on the document row).

Authorization: Owner-scoped document.

Request: No request body. Query parameter `token` (string).

Response: `FileResponse` with PDF binary content (`application/pdf`).

Error responses: 401/403 if token invalid or user lacks access; 404 if file missing.

API: POST /api/v1/documents/upload

Method: POST

Endpoint: `/api/v1/documents/upload`

Frontend caller: `DashboardPage.handleUpload()` → `api.postForm("/api/v1/documents/upload", formData)` — `frontend/src/app/dashboard/page.tsx`

Backend handler: `upload_document` — `backend/app/routes/documents.py`

Purpose: Accept a document file, create a `Document` record with `status = "queued"`, and enqueue the ingestion pipeline (Celery `process_document`, or synchronous `BackgroundTasks` fallback → `ingest_document`).

Authentication: Required.

Authorization: Owner-scoped — document row created for `user.id`; file stored under `UPLOAD_DIR/<user.id>/<filename>`.

Request: Multipart/form-data. No JSON body.

Fields (verified in handler): `file` (UploadFile), `user_id` (string), `original_name` (string), `document_id` (string UUID), optional `chunk_size`, `chunk_overlap`.

Response: `DocumentResponse` (202 Accepted) — the created document record including `task_id`.

```
{
  "id": "string",
  "original_name": "xolair_prescribing.pdf",
  "status": "queued",
  "processing_stage": "queued",
  "processing_progress": 0,
  "task_id": "celery-task-uuid"
}
```

Error responses: 400 validation (unsupported type/size), 401 unauthenticated, 413 body too large, 500 server error.

Execution flow: Frontend `handleUpload` → `api.postForm` → `upload_document` → DB insert → Celery task → response → UI adds document row with `status = "queued"`.

API Documentation

All endpoints below are mounted under the `/api/v1` prefix in `backend/app/main.py` and are called from the dashboard. Base URL configured in `frontend/src/lib/api.ts`: `NEXT_PUBLIC_API_URL` or `http://localhost:7860`. Every JSON request carries `Authorization: Bearer <token>` where token comes from `localStorage` (`api.getToken()`).

API: GET /api/v1/documents/

Method: GET

Endpoint: `/api/v1/documents/`

Frontend caller: `DocumentSidebar.fetchDocuments()` — `frontend/src/components/document/DocumentSidebar.tsx`

Backend handler: `list_documents` — `backend/app/routes/documents.py`

Purpose: List the authenticated user's active (non-trashed) documents, with optional search filter. Powers the document library shown on mount and after mutations.

Authentication: Required — `get_current_user` dependency.

Authorization: User-scoped — SQL filters `Document.user_id == user.id` and `Document.is_deleted.is_(False)`.

Request: No request body. Optional query parameters (verified via `_get_documents_query`): `q` (substring filter on `original_name`).

```
{}
```

Response: `DocumentListResponse` — object with `documents: DocumentResponse[]` and `total: int`. Each `DocumentResponse` (verified fields in `schemas.py`): `id`, `original_name`, `filename`, `file_size`, `page_count`, `chunk_count`, `status`, `error_message`, `uploaded_at`, `processing_progress`, `processing_stage`, `summary`, `keywords`, `extracted_urls`, `chunk_size`, `chunk_overlap`, `task_id`.

```
{
  "documents": [
    {
      "id": "string",
      "original_name": "xolair_prescribing.pdf",
      "file_size": 123456,
      "page_count": 8,
      "chunk_count": 14,
      "status": "ready",
      "error_message": null,
      "uploaded_at": "2026-01-01T00:00:00Z",
      "processing_progress": 100,
      "processing_stage": "completed",
      "summary": "...",
      "keywords": ["xolair", "dosing"],
      "extracted_urls": []
    }
  ],
  "total": 1
}
```

Error responses: 401 unauthenticated; 500 database error (global handler returns `ErrorResponse` envelope).

Execution flow: Frontend `DocumentSidebar` → `api.get("/api/v1/documents/")` → `list_documents` → SQLAlchemy `SELECT` → `DocumentListResponse` → frontend `setDocuments`.

Backend Dependency Table

Backend file	Function / class	Responsibility	Called by
backend/app/main.py	lifespan	Startup: init DB, dirs, embed model pre-load; shutdown: stop scheduler	FastAPI app lifecycle
backend/app/main.py	add_request_id middleware	Assigns X-Request-ID to every response	All dashboard requests
backend/app/auth.py	get_current_user	JWT validation + user resolution	All dashboard route handlers via Depends
backend/app/auth.py	decode_token	HS256 JWT decode → user_id	get_current_user
backend/app/database.py	get_db	Per-request SQLAlchemy session	Route handlers via Depends
backend/app/database.py	init_db	Create tables + migrations at startup	main.py lifespan
backend/app/routes/documents.py	list_documents	GET /documents/ — user's active documents	DocumentSidebar
backend/app/routes/documents.py	upload_document	POST /documents/upload — create row + enqueue ingestion	Page upload handler
backend/app/routes/documents.py	delete_document	Soft-delete document	DocumentSidebar trash
backend/app/routes/documents.py	restore_document	Restore soft-deleted document	DocumentSidebar trash tab
backend/app/routes/documents.py	permanent_delete_document	Hard-delete document + chunks	DocumentSidebar trash tab
backend/app/routes/documents.py	list_trash_documents	List soft-deleted documents	DocumentSidebar trash tab
backend/app/routes/documents.py	serve_pdf	Stream PDF file bytes with token auth	PDFViewer
backend/app/routes/documents.py	update_chunk_settings	Update chunk size/overlap + re-ingest	DocumentSettings
backend/app/routes/documents.py	_deserialize_doc	Parse extracted_urls JSON on Document rows	Document list/detail responses
backend/app/services/document_ingestion.py	ingest_document	Full ingestion pipeline (chunk → embed → store → summarize)	process_document task / upload fallback
backend/app/tasks.py	process_document	Celery task wrapping ingestion with retries	upload_document (Celery delay)
backend/app/routes/chat.py	ask_question_stream	POST /chat/ask/stream — SSE RAG answer	ChatPanel
backend/app/routes/chat.py	generate_answer_stream	Stream wrapper delegating to app.rag.llm.agent	ask_question_stream
backend/app/routes/chat.py	generate_answer	Non-streaming RAG answer	ask_question_stream (non-stream path)
backend/app/routes/chat.py	list_sessions / create_session / rename_session / delete_session	Chat session CRUD	chat-store
backend/app/routes/chat.py	get_session_history	Messages for a session	chat-store
backend/app/rag/llm/agent.py	generate_answer_stream	RAG stream: retrieve → context → LLM → SSE chunks	chat.py wrapper
backend/app/rag/llm/agent.py	_generate_answer_stream_direct	Ollama/Bedrock direct RAG stream	generate_answer_stream
backend/app/rag/llm/agent.py	_format_context	Build context string from retrieved chunks	_generate_answer_*_direct

Backend file	Function / class	Responsibility	Called by
backend/app/rag/retrieval/retriever.py	retriever	Hybrid retrieval: embed + vector + BM25 + RRF + rerank	agent.py
backend/app/rag/retrieval/transform_query.py	transform_query	Query + up to 3 paraphrase variants	retrieve
backend/app/rag/retrieval/rrf_merge.py	rrf_merge	Reciprocal Rank Fusion of vector + BM25 lists	retrieve
backend/app/rag/embeddings/embed_query.py	embed_query	Embed a query string (768-d, cached)	retrieve
backend/app/rag/embeddings/get_embedding_model.py	get_embedding_model	Singleton OllamaEmbeddings (nomic-embed-text)	embed_query, indexing
backend/app/rag/vectordb/query_chunks.py	query_chunks	Chroma similarity search with document_id filter	retrieve
backend/app/rag/vectordb/store_chunks.py	store_chunks	Embed + store chunks in Chroma + rebuild BM25	ingest_document
backend/app/rag/vectordb/delete_store.py	delete_document_chunks	Remove Chroma + BM25 entries for a document	Document deletion
backend/app/rag/llm/ollama_client.py	OllamaChatClient.chat_completion	POST /api/chat to Ollama	agent direct path
backend/app/rag/llm/ollama_embeddings.py	OllamaEmbeddings	LangChain-compatible Ollama embeddings via /api/embed	get_embedding_model
backend/app/cache.py	get_cached_payload / set_cached_response	Redis/LRU cache of RAG answers	ask_question_stream
backend/app/routes/auth.py	read_users_me / dev_login / refresh_token / update_hf_token / google-drive endpoints	Auth + profile + Drive status	Frontend auth store / DocumentSidebar
backend/app/exceptions.py	AppException hierarchy	Typed errors (401/403/404/409/429/502) with code/message/details	Route handlers raise; global handler renders ErrorResponse

Backend Architecture

Entry point

- backend/app/main.py — FastAPI app. lifespan startup runs init_db() (creates tables), ensures UPLOAD_DIR and CHROMA_PERSIST_DIR exist, pre-loads the embedding model. Shutdown stops the scheduler.
- Middleware (in order added): StructuredLoggingMiddleware (outermost), request-ID middleware, CORS, trusted host, rate limit (SlowAPI), body-size limit.
- Routes mounted under prefix /api/v1:
  - auth_router (backend/app/routes/auth.py)
  - documents_router (backend/app/routes/documents.py)
  - chat_router (backend/app/routes/chat.py)
  - github_router, admin_router, workspaces_router, health_router, profile_router

Authentication dependency

- get_current_user (backend/app/auth.py) — FastAPI Depends used by every dashboard-backed route. Extracts JWT from Authorization: Bearer header or access_token cookie; rejects API keys unless prefixed pdf_rag_; decodes HS256 JWT via decode_token; loads User from DB; sets request.state.user_id and observability context var.

- Admin-only dependency `get_admin_user` / `get_current_admin` — not used by dashboard routes.

Route handlers used by the dashboard

Route (mounted under `/api/v1`)	File	Handler	Dashboard caller
`GET /documents/`	`routes/documents.py`	`list_documents`	DocumentSidebar `fetchDocuments`
`POST /documents/upload`	`routes/documents.py`	`upload_document`	Page `handleUpload`
`PATCH /documents/{document_id}`	`routes/documents.py`	`update_document` (rename/chunk settings split)	DocumentSidebar rename; DocumentSettings chunk settings
`DELETE /documents/{document_id}`	`routes/documents.py`	`delete_document` (soft delete)	DocumentSidebar trash
`POST /documents/{document_id}/restore`	`routes/documents.py`	`restore_document`	DocumentSidebar trash tab
`DELETE /documents/{document_id}/permanent`	`routes/documents.py`	`permanent_delete_document`	DocumentSidebar trash tab
`GET /documents/trash`	`routes/documents.py`	`list_trash_documents`	DocumentSidebar trash tab
`GET /documents/{document_id}/pdf`	`routes/documents.py`	`serve_pdf`	PDFViewer via `api.getPdfUrl`
`POST /documents/{document_id}/chunk_settings`	`routes/documents.py`	`update_chunk_settings`	DocumentSettings modal
`GET /auth/me`	`routes/auth.py`	`read_users_me`	auth-store `initializeAuth`
`POST /auth/dev-login`	`routes/auth.py`	`dev_login`	auth-store dev bypass
`POST /auth/refresh`	`routes/auth.py`	`refresh_token`	ApiClient auto-refresh on 401
`POST /auth/logout`	`routes/auth.py`	`logout`	auth-store logout / Header
`PUT /auth/hf-token`	`routes/auth.py`	`update_hf_token`	Header HF dialog / auth-store `setHfToken`
`GET /auth/google-drive/status`	`routes/auth.py`	`google_drive_status`	DocumentSidebar
`POST /auth/google-drive/connect`	`routes/auth.py`	`google_drive_connect`	DocumentSidebar
`DELETE /auth/google-drive/disconnect`	`routes/auth.py`	`google_drive_disconnect`	DocumentSidebar
`GET /chat/sessions`	`routes/chat.py`	`list_sessions`	chat-store `fetchSessions`
`POST /chat/sessions`	`routes/chat.py`	`create_session`	chat-store `createSession`
`PUT /chat/sessions/{id}`	`routes/chat.py`	`rename_session`	chat-store `renameSession`
`DELETE /chat/sessions/{id}`	`routes/chat.py`	`delete_session`	chat-store `deleteSession`
`GET /chat/history/session/{id}`	`routes/chat.py`	`get_session_history`	chat-store `fetchSessionHistory`
`POST /chat/ask/stream`	`routes/chat.py`	`ask_question_stream` (SSE)	ChatPanel `handleSend`
`WS /chat/ws`	`routes/chat.py`	`chat_websocket`	ChatPanel fallback (attempted first)

Services and business logic invoked

- `app/services/document_ingestion.py` — `ingest_document()`: page count → chunk → embed → store vectors (Chroma) + BM25 → keywords → summary → URL extraction; updates `Document` row status/progress.

- `app/tasks.py` — Celery task `process_document`: wraps `ingest_document`, records retries, marks `failed` on exhaustion.
- `app/rag/llm/agent.py` — `generate_answer_stream()` / `generate_answer()`: orchestrate retrieval → prompt → LLM; Ollama direct path or HF ReAct or Bedrock.
- `app/rag/retrieval/retriever.py` — `retrieve()`: query transformation, embedding, vector + BM25 search, RRF merge, optional rerank, confidence scoring.
- `app/rag/embeddings/embeddings.py` — `embed_query()` / `get_embedding_model()`: Ollama `nomic-embed-text` (768-d) with LRU/Redis cache.
- `app/rag/vectordb/vectorstore.py` — `query_chunks()` / `store_chunks()`: Chroma per-user collection, cosine similarity.
- `app/rag/llm/ollama_client.py` — `OllamaChatClient.chat_completion()`: POST to `http://localhost:11434/api/c`
- `app/cache.py` — `get_cached_payload` / `set_cached_response`: Redis or LRU cache of RAG answers by (user, document, question).
- `app/database.py` — SQLAlchemy sync engine (SQLite default), `get_db()` dependency, `get_db_session()` context manager for background tasks.

Frontend State Management

Store	Source	Persistence	Key state relevant to dashboard
`auth-store`	`frontend/src/store/auth-store.ts` (Zustand)	localStorage (`token`, `refresh_token`)	`user`, `token`, `loading`, `initialized`
`chat-store`	`frontend/src/store/chat-store.ts` (Zustand)	Server + local only (attachments in localStorage)	`messages`, `input`, `streaming`, `sessions`, `activeSessionId`, `historyLoading`
`notes-store`	`frontend/src/store/notes-store.ts` (Zustand `persist`)	localStorage `genova-shared-notes` (`html` only)	`html`, `notesOpen`, `focusToken`
`workspace-store`	`frontend/src/store/workspace-store.ts` (Zustand)	None (default "personal")	`workspace`

Page-local state (DashboardPage)

- `documents: DocInfo[]` — `useState`, set by `loadDocuments()` and after upload/delete.
- `activeDoc: DocInfo | null` — selected document.
- `viewerOpen`, `notesOpen`, `compareOpen`, `showPdfDock`, `showInspector` — UI visibility booleans.
- `pdfPage`, `pdfHighlightTarget` — citation navigation.
- `activeInspectorTab: "notes" \| "pdf"`.
- `prevDocsRef` — `useRef` to detect document list changes and reload chat.

Data fetching

- No server-side data fetching (client page, `ssr: false` for lazy components).
- `loadDocuments()` runs in `useEffect` on mount and when `documents` change; no SWR/React Query — direct `api.get` calls with manual refresh.
- No client-side caching of API responses at the page level; caching exists server-side (`app/cache.py`) for RAG answers.
- Loading states: `uploading`, `uploadProgress`, `deleting`, `bulkDeleting`, `historyLoading` (chat store), `streaming` (chat store).
- Error states: `uploadError`, `driveError`, toast notifications, `sonner` toasts.

Authentication context

- `AuthProvider` (`frontend/src/lib/auth.tsx`) calls `initializeAuth()` on mount and listens to `auth:tokens-refreshed` / `auth:logged-out` window events dispatched by `ApiClient` on token refresh /

logout.
- `api.getToken()` reads `localStorage.getItem("token")`; attached as `Authorization: Bearer <token>` on every request unless token is `"cookie"` (OAuth cookie flow).

CompareView elements

Element	Purpose	User interaction
Doc pickers (Left/Right)	Choose two ready PDFs	Dropdown → set left/right doc
Side-by-side pages	Two PDF.js viewers with independent page state	Page navigation per pane
Close button	Exit compare	`onClose`

Header elements (dashboard shell)

Element	Purpose	User interaction
Workspace switcher	personal / company	Select → `setWorkspace`
Theme toggle	light/dark/ocean/forest/sunset	Click → `setTheme`
Logout button	Sign out	Click → `logout()` + router replace
HF token dialog	Save/remove HuggingFace token	Input + Save/Remove → `PUT /api/v1/auth/hf-token`

Empty / error / loading states (verified)

- **Dashboard initial:** renders `null` until `initialized` (auth gate); no skeleton shell enforced — auth-store fail-safe timeout prevents infinite loading.
- **Document list empty:** no distinct empty component in DocumentSidebar beyond tabs showing no rows.
- **Trash empty:** no rows shown when trash list empty.
- **Compare empty:** "Select a PDF for the left/right pane" empty panes.
- **PDF viewer error:** "Failed to load PDF" error UI from react-pdf `onLoadError`.
- **PDF viewer no data:** "No PDF document selected" when `noData`.
- **Upload error:** `uploadError` state shown in DocumentSidebar; toast on failure.
- **Connection error:** `ApiClient` fetch TypeError → `CONNECTION_ERROR_MESSAGE` ("Could not connect to the server…").
- **Chat errors:** SSE `error` event rendered; toast on send failure.

User Interactions

1. Open dashboard

1. User: browser requests http://localhost:3000/dashboard (or is redirected after Google/HF OAuth with `?token=` & `?refresh_token=` in URL).
2. Component: `DashboardPage` mounts; `useAuth()` → auth-store `initializeAuth()`.
3. Handler: `initializeAuth()` reads `localStorage.token`; if absent and dev bypass on localhost, calls `POST /api/v1/auth/dev-login`; otherwise calls `GET /api/v1/auth/me`.
4. API: `GET /api/v1/auth/me` (or dev-login).
5. Backend: validates JWT (or creates dev user), returns `UserResponse`.
6. UI: `initialized = true`, page renders desk; `loadDocuments()` fires.

2. Load document library

1. Component: `DocumentSidebar` mount → `useEffect` calls `fetchDocuments()`.
2. Handler: `fetchDocuments()` → `api.get("/api/v1/documents/")` (+ `api.get("/api/v1/auth/google-drive/status")`
3. Backend: `GET /api/v1/documents/` → SQL `SELECT` from `documents` where `user_id` and `is_deleted = false`; returns `DocumentListResponse`.
4. UI: rows render with status badges; skeletons while `loading`.

3. Upload a document

1. User: drops files on page or clicks upload in DocumentSidebar (accepted: pdf, docx, txt, md).
2. Component: `DashboardDropOverlay.onDrop` / DocumentSidebar upload handler → `onDrop(files)` → `handleUpload(files)` in page.
3. Handler: `FormData` with `file`, `user_id`, `original_name`, `document_id`, optional `chunk_size`/`chunk_overlap` → `api.postForm("/api/v1/documents/upload", formData)`.
4. Backend: `POST /api/v1/documents/upload` → validates file, creates `Document` row (`status = "queued"`), enqueues Celery `process_document` (or fallback `BackgroundTasks` → `ingest_document`), returns `DocumentResponse` (202).
5. UI: upload progress bar; `loadDocuments()` refreshes list; new doc appears with status; on completion, status → `ready`.

4. Select a document

1. User: clicks a document row.
2. Component: `DocumentSidebar` → `onSelectDoc(doc)` → page `setActiveDoc(doc)`.
3. UI: status bar updates (DocumentStatusBar chips: status, pages, chunks, uploaded date); inspector tab content bound to `activeDoc`.

5. Ask a question (RAG chat)

1. User: types question in ChatPanel textarea; optionally selects model provider (ollama/bedrock) and attaches docs; clicks Send or presses Enter.
2. Component: `ChatPanel.handleSend()`.
3. Handler: appends user message optimistically; calls `api.streamPost("/api/v1/chat/ask/stream", { question, document_id, document_ids, top_k, llm_provider })` (WebSocket `/api/v1/chat/ws` attempted first, falls back to SSE).
4. Backend: `POST /api/v1/chat/ask/stream` → validates input, saves user message, checks response cache (`app/cache.py`), runs `generate_answer_stream(...)` → `retrieve(...)` (embed + Chroma + BM25 + RRF + optional rerank) → builds prompt → Ollama/Bedrock chat completion; streams SSE events (`sources`, `token`, `usage`, `done`); saves assistant message.
5. UI: assistant message streams token-by-token; source chips appear; citations clickable → opens PDF viewer at page + highlight rect.

6. Open citation in PDF

1. User: clicks a citation in a chat message.
2. Component: `ChatPanel.onCitationClick(target)` → sets `pdfPage`, `pdfHighlightTarget`, `viewerOpen = true`, `activeInspectorTab = "pdf"`.
3. UI: inspector opens PDF tab; `PDFViewer` renders `api.getPdfUrl(documentId)` → `GET /api/v1/documents/{id}/pdf?tol` with highlight overlay.

7. Rename / delete / restore document

1. User: clicks rename/trash/restore/delete-forever on a row (or bulk actions).
2. Component: `DocumentSidebar` handlers.
3. API: `api.renameDocument(id, name)` → `PATCH /api/v1/documents/{id}`; `api.delete("/api/v1/documents/{id}")` → soft delete; `api.post(.../restore)`; `api.delete(.../permanent)`.
4. UI: list refreshes via `onDocumentsChange()`; toasts confirm.

8. Change chat session

1. User: clicks a session in ChatSessionSidebar.
2. Component: `handleSelectSession(id)` → `setActiveSessionId(id)` + `fetchSessionHistory(id)` → `GET /api/v1/chat/history/session/{id}`.
3. UI: messages replaced with history.

ChatPanel elements

Element	Purpose	User interaction
Model provider dropdown	`ollama` vs `bedrock`	Select → `setLlmProvider`
Attach button	Attach documents to query	Click → attach picker; `/attach` command
Attached-doc chips	List attached docs with remove	Click X → `removeAttachedDoc`
Textarea input	Type question	Enter → `handleSend`
Send button	Submit question	Click → `handleSend` (disabled while streaming or no ready docs)
Stop button	Cancel stream	Click → `handleStop` (abort controller)
Message bubbles	User/assistant messages with sources	Click citation → `onCitationClick` (opens PDF at page + highlight)
Feedback up/down	Rate assistant message	Click (stored locally/UI only)
Quiz card	Quiz-intent question UI	Rendered when `quiz` payload present

PDFViewer elements

Element	Purpose	User interaction
Page render	`react-pdf <Page>`	—
Zoom controls	Zoom in/out/reset	Buttons set `scale`
Rotation	Rotate page	Button sets `rotation`
Page input	Jump to page	Type + Enter
Search	Search text in PDF	`usePdfSearch` → highlight rects
Citation overlays	Yellow rects from RAG sources	Rendered from `highlightTarget.rects`
Load/error/empty states	Loader, “Failed to load PDF”, “No PDF document selected”	Automatic

DocumentSidebar elements

Element	Purpose	User interaction	Inputs / output
Document list tabs (“Active” / “Trash”)	Switch between active and soft-deleted documents	Click tab	`activeTab` state
Search input	Filter documents by name	Type	`q` passed to `GET /api/v1/documents/`
Upload button / drop zone	Upload pdf/docx/txt/md	Click or drop files	`POST /api/v1/documents/upload` (multipart)
Upload progress bar	Show upload progress	—	`uploadProgress` state
Document row	Select document, show status + size	Click row → `onSelectDoc`	Highlights active doc
Status badge	`ready` / `processing` / `failed` / `pending`	—	Color-coded pill
Rename (inline edit)	Edit document `original_name`	Click edit icon, type, save	`PATCH /api/v1/documents/{id}` via `api.renameDocument`
Trash icon	Soft-delete document	Click	`DELETE /api/v1/documents/{id}`
Restore icon (trash tab)	Restore soft-deleted doc	Click	`POST /api/v1/documents/{id}/restore`

Element	Purpose	User interaction	Inputs / output
Delete forever icon (trash tab)	Permanent delete	Click	`DELETE /api/v1/documents/{id}/permanent`
Bulk action bar	Bulk delete / restore / permanent delete	Select checkboxes + action button	Loops `api.delete` / `api.post(.../restore)` / `api.delete(.../permanent)`
Settings button	Open chunk settings modal	Click	`POST /api/v1/documents/{id}/chunk_sett`
Google Drive connect	Connect Drive folder	Click	`POST /api/v1/auth/google-drive/connect`
Google Drive disconnect	Disconnect Drive	Click	`DELETE /api/v1/auth/google-drive/disconn`
Drive status indicator	Show Drive connected state	—	`GET /api/v1/auth/google-drive/status`

ChatSessionSidebar elements

Element	Purpose	User interaction
Session list	List of chat sessions	Click session → select + fetch history
New session button	Create session	Click → `POST /api/v1/chat/sessions`
Rename input	Rename session	Edit + save → `PUT /api/v1/chat/sessions/{id}`
Delete button	Delete session	Click → `DELETE /api/v1/chat/sessions/{id}`
Mobile FAB / overlay	Open sessions on mobile	Toggle overlay

UI Functionality

All elements below are verified in `frontend/src/app/dashboard/page.tsx` and its imported components. No elements are invented.

Page-level elements (DashboardPage)

Element	Purpose	User interaction	Notes
Auth gate	`useAuth()` returns `user`, `initialized`. While `initialized === false` or no user, page renders null (no dashboard shown).	None — automatic redirect to `/login` on unauthenticated state	Guard at top of `DashboardPage`
Document list state	`documents: DocInfo[]` fetched on mount and after mutations	—	Drives DocumentSidebar and ChatPanel `documents` prop
Active document	`activeDoc: DocInfo | null` — selected document	Click document row in DocumentSidebar	Drives status bar, inspector tabs, PDF viewer
Viewer toggle	`viewerOpen` state	Header viewer button / citation click opens PDF viewer	Controls inspector PDF tab
Notes toggle	`notesOpen` state	Header notes button	Controls inspector notes tab

Element	Purpose	User interaction	Notes
Compare toggle	`compareOpen` state	Header compare button	Renders CompareView inspector
Inspector tab	`activeInspectorTab: "notes" | "pdf"`	Tab buttons in inspector header	Switches NotesPanel vs PDFViewer
Drop overlay	`isDraggingOver` from `useDashboardDrop`	Drag files over window	Full-page overlay; `onDrop` → `handleUpload`
Toasts	`sonner`	Automatic on upload/delete/errors	Connection error banner

Component hierarchy

```
DashboardPage (page.tsx, client)
  AuthProvider (context) - user, initialized state from localStorage
  Header - workspace switcher, theme, logout, HF token dialog
  genova-desk stage
      DocumentSidebar - docs list, upload, trash, drive status
      ChatSessionSidebar - sessions
      ChatPanel - RAG chat (streaming SSE)
      Inspector (Notes / PDF / Compare)
          NotesPanel (notes editor)
          PDFViewer (PDF.js)
          CompareView (two PDFs)
  DashboardDropOverlay (drag-over full page)
```

Frontend file inventory

File	Responsibility	Dashboard usage
`frontend/src/app/dashboard/page.tsx`	Main dashboard page component; document list state, upload, selection, chat/inspector orchestration	Primary route component; mounts all child UI
`frontend/src/app/dashboard/dashboard.css`	Dashboard visual design tokens and layout	Styles all dashboard regions
`frontend/src/components/layout/Header.tsx`	Top bar: auth user, workspace, theme, logout, HF token	Persistent shell of the dashboard
`frontend/src/components/document/DocumentSidebar.tsx`	Document list, upload, rename, soft-delete, restore, permanent delete, trash, Google Drive status	Left sidebar document management
`frontend/src/components/document/DocumentCard.tsx`	Individual document row with status badge and actions	Row within DocumentSidebar
`frontend/src/components/document/BulkActionBar.tsx`	Multi-select toolbar in DocumentSidebar	Multi-select delete/restore/permanent-delete controls
`frontend/src/components/document/ChunkSettings.tsx`	Chunk size/overlap settings per document (`POST .../chunk_settings`)	Settings modal invoked from DocumentSidebar
`frontend/src/components/document/PDFViewer.tsx`	PDF.js page rendering, zoom, rotation, search, RAG citation overlay rects	Inspector tab; opened from chat citations
`frontend/src/components/document/DashboardDropOverlay.tsx`	Full-page drag-over indicator (delegates `onDrop` to page)	Drag-and-drop upload affordance
`frontend/src/components/document/CompareView.tsx`	Side-by-side two-PDF comparison with page sync	Inspector compare mode

File	Responsibility	Dashboard usage
`frontend/src/components/chat/ChatSessionSidebar.tsx`	Chat session list: create, rename, delete, select	Right sidebar session management
`frontend/src/components/chat/ChatPanel.tsx`	RAG chat: input, model provider select, attachments, SSE stream rendering, citations, quiz	Central chat area
`frontend/src/components/chat/MessageBubble.tsx`	Single message render with sources and feedback	Inside ChatPanel
`frontend/src/components/chat/SourceCard.tsx`	Citation chunk card with page + highlight rects	Inside ChatPanel messages
`frontend/src/components/chat/QuizCard.tsx`	Quiz UI for quiz-intent questions	Inside ChatPanel
`frontend/src/components/notes/NotesPanel.tsx`	Shared notes: editor, export to PDF/TXT/MD, drag-in messages	Inspector notes tab
`frontend/src/components/notes/NotesEditor.tsx`	CKEditor 5 rich-text editor	Inside NotesPanel
`frontend/src/components/graph/KnowledgeGraph.tsx`	Knowledge graph visualization (xyflow)	Lazy-loaded panel (imported; not core to verified flow)
`frontend/src/components/dashboard/DashboardPanel.tsx`	Dashboard panel shell (FadeIn + SpotlightCard)	Defined; not imported by page.tsx
`frontend/src/components/dashboard/DashboardStatusBar.tsx`	Status bar: counts (status, pages, chunks, date)	Defined; not imported by page.tsx
`frontend/src/hooks/useDashboardDrop.ts`	Window dragenter/dragover/drop tracking; accepted extensions pdf/docx/txt/md	Powers DashboardDropOverlay
`frontend/src/hooks/usePdfSearch.ts`	In-viewer text search (PDF.js text layer)	Powers PDFViewer search highlights
`frontend/src/lib/api.ts`	`ApiClient`: GET/POST/PUT/DELETE/postForm/streamPost, Bearer token from localStorage, auto 401 refresh, SSE parsing	All dashboard API calls
`frontend/src/lib/auth.tsx`	`AuthProvider` + `useAuth` (Zustand auth store wrapper)	Authentication gate on the page
`frontend/src/lib/dev-auth.ts`	`isDevAuthBypass()` — localhost auto sign-in gate	Controls dev-login path
`frontend/src/store/auth-store.ts`	Zustand auth store: login, dev-login, /me, refresh, logout, hf-token	`user`, `initialized` consumed by page
`frontend/src/store/chat-store.ts`	Zustand chat store: sessions, messages, history, SSE message types	ChatPanel state
`frontend/src/store/notes-store.ts`	Zustand notes store (persist: localStorage `genova-shared-notes`)	NotesPanel state
`frontend/src/store/workspace-store.ts`	Workspace selector (personal/company)	Header workspace switcher

Frontend Architecture

Route and page structure

- **Route definition:** Next.js App Router static route `/dashboard` implemented in `frontend/src/app/dashboard/page.tsx` (`"use client"`). Next config `frontend/next.config.ts` sets `output: "export"` (static export).
- **Page component:** `DashboardPage` in `frontend/src/app/dashboard/page.tsx`. Imported layout/child components:
  - `Header` (`@/components/layout/Header`) — top strip: workspace switcher, theme toggle, logout, HF token dialog
  - `DocumentSidebar` (`@/components/document/DocumentSidebar`) — document library + upload + trash

  - `ChatSessionSidebar` (`@/components/chat/ChatSessionSidebar`) — chat session list
  - `ChatPanel` (`@/components/chat/ChatPanel`) — RAG chat with streaming + sources
  - `CompareView` (`@/components/document/CompareView`) — two-PDF side-by-side comparison
  - `DashboardDropOverlay` (`@/components/document/DashboardDropOverlay`) — full-page drag-over overlay
  - `NotesPanel` (`@/components/notes/NotesPanel`, lazy via `next/dynamic`, `ssr: false`) — shared notes
  - `PDFViewer` (`@/components/document/PDFViewer`, lazy via `next/dynamic`, `ssr: false`) — PDF.js viewer
  - `KnowledgeGraph` (`@/components/graph/KnowledgeGraph`, lazy via `next/dynamic`, `ssr: false`) — knowledge graph panel (referenced in imports; graph wiring not part of core dashboard flow)
- **Parent layout:** `frontend/src/app/layout.tsx` (root layout) wraps the page with `ThemeProvider` → `MuiProvider` → `AuthProvider` → `I18nProvider` → `TooltipProvider`.
- **Page stylesheet:** `frontend/src/app/dashboard/dashboard.css` — design tokens (`--gd-ground`, `--gd-surface`, `--gd-border`, `--gd-primary`, status colors), layout grid (docs 240px + sessions 280px + inspector 360px + chat 1fr), dark theme, status pill colors.